To understand how Python resolves Method Resolution Order (MRO) without getting lost in mathematical formulas, you only need to remember **two simple rules**:

> **Left-to-Right Priority:** Python checks parents in the order they are written in the class definition (e.g., for `class A(B, C)`, it looks at `B` before `C`).
>
> **The "Unvisited Child" Rule:** A parent class **cannot** be added to the MRO list until **ALL of its child classes** have already been added.

Step-by-Step MRO Derivation

Let's look at the class declarations and their parent-child relationships:

- `A(B, C)` $\rightarrow$ `A` is a child of `B` and `C`
- `B(E, D)` $\rightarrow$ `B` is a child of `E` and `D`
- `C(D, F)` $\rightarrow$ `C` is a child of `D` and `F`
- `D(Base)`, `E(Base)`, `F(Base)`

Step 1: Start with `A`

- **MRO List:** `[A]`

Step 2: Move to `A`'s first parent, `B`

- **Check `B`:** Has all of `B`'s children been visited? Yes, `A` is its only child and `A` is already in our list.
- **MRO List:** `[A, B]`

Step 3: Move to `B`'s first parent, `E`

- **Check `E`:** Has all of `E`'s children been visited? Yes, `B` is its child and `B` is already in our list.
- **MRO List:** `[A, B, E]`

Step 4: Move to `B`'s second parent, `D` — *The Key Moment!*

- **Check `D`:** Can we add `D` now?
- **NO!** Look at `C(D, F)`. Class `C` is also a child of `D`. But **`C` has not been visited yet!**
- If we add `D` now, the parent (`D`) would appear before its child (`C`), breaking Rule #2.
- Therefore, **`D` is put on hold (blocked)** until `C` is processed.

Step 5: Switch branches to `A`'s second parent, `C`

- **Check `C`:** Has all of `C`'s children been visited? Yes, `A` is already in our list.
- **MRO List:** `[A, B, E, C]`

Step 6: Move to `C`'s first parent, `D`

- **Check `D` again:** Now that `C` is in our list, all children of `D` (`B` and `C`) have been visited! `D` is no longer blocked.
- **MRO List:** `[A, B, E, C, D]`

Step 7: Move to `C`'s second parent, `F`

- **Check `F`:** Its child `C` is already visited.
- **MRO List:** `[A, B, E, C, D, F]`

Step 8: Add the remaining root classes

- All paths lead to `Base` and finally Python's built-in `object`.

Final `A.mro()` Result:

Python

```
[A, B, E, C, D, F, Base, object]
```

How `a.show()` Executes Line-by-Line

When you call `a.show()`, `super()` **does not look at the immediate parent of the class written in code**. Instead, `super()` looks at the `A.mro()` **list above** and moves to the **next item in that list**.

Plaintext

```
A.mro() position:   [1] -> [2] -> [3] -> [4] -> [5] -> [6] -> [7]
Class executed:      A  ->  B  ->  E  ->  C  ->  D  ->  F  -> Base
```

`A.show()` executes:
- Prints: `"A"`
- `super().show()` looks up next in MRO $\rightarrow$ **`B`**

`B.show()` executes:
- Prints: `"B"`
- `super().show()` looks up next in MRO $\rightarrow$ **`E`**

`E.show()` executes:
- Prints: `"E"`
- `super().show()` looks up next in MRO $\rightarrow$ **`C`** *(Notice `E` delegates to `C` because `C` is next in `A`'s MRO!)*

`C.show()` executes:
- Prints: `"C"`
- `super().show()` looks up next in MRO $\rightarrow$ **`D`**

`D.show()` executes:

- Prints: `"D"`
- `super().show()` looks up next in MRO $\rightarrow$ **`F`**

`F.show()` executes:

- Prints: `"F"`
- `super().show()` looks up next in MRO $\rightarrow$ **`Base`**

`Base.show()` executes:

- Prints: `"Base"`
- *(No `super().show()` call here, execution finishes).*

Exact Console Output

Plaintext

```
A
B
E
C
D
F
Base
```